SIMATS ENGINEERING

SIMATS Online Programming Lab

JAVA PROGRAMMING

MD Imthiaz I
192521360RD

CO2-AT1-Scenario Based Assessment

Back

QUESTIONS

Q1
Smart Hospital Patient Registration System
20 marks

Q2
Online Library Book Management System
20 marks

2/2 answered

Q2 **Online Library Book Management System** 20 marks

Scenario

Online Library Book Management System

A university library is planning to automate its book management process. The librarian wants a Java application that stores book details, prevents duplicate book entries, allows students to search for books quickly, and generates category-wise reports. The application should use Java String handling and Java Collection Framework to efficiently manage the library records.

Requirements

Each book contains the following details:

Book ID

Book Title

Author Name

Category

Publisher

Use:

ArrayList to store all book objects.

HashMap to store Book ID as the key for quick searching.

HashSet to maintain unique author names.

Menu Driven Program

1. Add Book

2. Search Book by ID
3. Display All Books
4. Display Unique Authors
5. Category-wise Book Count
6. Exit

Requirement 1 – Add Book (4 Marks)

Accept

Book ID

Book Title

Author Name

Category

Publisher

Validate the following:

Book ID must be unique.

Remove leading and trailing spaces.

Book Title should be stored in Title Case.

Category should be converted to uppercase.

Publisher name should not be empty.

Store the record in both

ArrayList

HashMap

Requirement 2 – Search Book (3 Marks)

Accept Book ID.

If found, display

Book ID

Book Title

Author

Category

Publisher

Otherwise,

Book Not Available

Requirement 3 – Display All Books (4 Marks)

Display all records in tabular format.

Example

```
-----------------------------------------------------------------
-----
ID TITLE AUTHOR CATEGORY
-----------------------------------------------------------------
-----
```

B101 Java Programming Herbert Schildt PROGRAMMING
B102 Data Structures Mark Allen COMPUTER SCIENCE

Requirement 4 – Display Unique Authors (3 Marks)
Display all distinct author names using HashSet.
Example
Unique Authors

Herbert Schildt

James Gosling

Yashavant Kanetkar

Mark Allen

Requirement 5 – Category-wise Book Count (4 Marks)
Display the number of books available in each category using
HashMap<String,Integer>
Example
PROGRAMMING : 12

DATABASE : 8

NETWORKING : 6

Requirement 6 – String Operations (2 Marks)
For every book,
Display
Length of Book Title
Number of words in the title
Category in uppercase
Example
Input
Java Programming Language
Output
Length : 25

Words : 3

Category : PROGRAMMING

Sample Input

1

B101

Java Programming

Herbert Schildt

Programming

McGraw Hill

1

B102

Data Structures

Mark Allen

Computer Science

Pearson

3

4

5

2

B101

6

Sample Output

Book Added Successfully.

Book Added Successfully.

Book ID Title Author

B101 Java Programming Herbert Schildt

B102 Data Structures Mark Allen

Unique Authors

Herbert Schildt

Mark Allen

Category-wise Count

PROGRAMMING : 1

COMPUTER SCIENCE : 1

Book Details

Book ID : B101

Title : Java Programming

Author : Herbert Schildt

Category : PROGRAMMING

Publisher : McGraw Hill

Constraints

Maximum books = 500

Duplicate Book IDs are not allowed.

Arrays are not allowed.

Use only Java Collections.

Book search should be performed using HashMap.

Use HashSet for maintaining unique authors.

Your Code

```
import java.util.*;
public class Main {
    private static String toTitleCase(Str
        if (text == null || text.trim().i
        String[] words = text.trim().spli
        StringBuilder titleCase = new Str
        for (int i = 0; i < words.length;
            if (!words[i].isEmpty()) {
                titleCase.append(Characte
                          .append(words[i
                if (i < words.length - 1)
                    titleCase.append(" ")
                }
            }
        }
        return titleCase.toString();
    }
    public static void main(String[] args
        Scanner sc = new Scanner(System.i
```

```java

        List<Book> bookList = new ArrayLi
        Map<String, Book> bookMap = new H
        Set<String> authorSet = new Linke

        while (sc.hasNextLine()) {
            String choiceLine = sc.nextLi
            if (choiceLine.isEmpty()) cor

            int choice;
            try {
                choice = Integer.parseInt
            } catch (NumberFormatExceptic
                continue;
            }

            if (choice == 1) {

                if (!sc.hasNextLine()) br
                String id = sc.nextLine()

                if (!sc.hasNextLine()) br
                String title = sc.nextLir

                if (!sc.hasNextLine()) br
                String author = sc.nextLi

                if (!sc.hasNextLine()) br
                String category = sc.next

                if (!sc.hasNextLine()) br
                String publisher = sc.nex
                if (bookMap.containsKey(i
                    continue;
                }

                title = toTitleCase(title

                Book book = new Book(id,
                bookList.add(book);
                bookMap.put(id, book);
                authorSet.add(author);

                System.out.println("Book

            } else if (choice == 2) {
                if (!sc.hasNextLine()) br
                String searchId = sc.next
```

```

                if (bookMap.containsKey(s
                    Book b = bookMap.get(
                    System.out.println("E
                    System.out.println("E
                    System.out.println("T
                    System.out.println("A
                    System.out.println("C
                    System.out.println("P
                } else {
                    System.out.println("E
                }

            } else if (choice == 3) {
                System.out.println("-----
                System.out.printf("%-10s
                System.out.println("-----
                for (Book b : bookList) {
                    System.out.printf("%-
                }

            } else if (choice == 4) {
                System.out.println("Uniqu
                for (String author : auth
                    System.out.println(au
                }

            } else if (choice == 5) {
                System.out.println("Categ
                Map<String, Integer> cate
                for (Book b : bookList) {
                    categoryCount.put(b.c
                }
                for (Map.Entry<String, In
                    System.out.println(en
                }

            } else if (choice == 6) {

                break;
            }
        }
        sc.close();
    }
}
class Book {
    String id;
    String title;
```

```
    String author;
    String category;
    String publisher;

    public Book(String id, String title,
        this.id = id;
        this.title = title;
        this.author = author;
        this.category = category;
        this.publisher = publisher;
    }
}
```

Input

```
1
B101
Java Programming
Herbert Schildt
```

Output

```
Book Added Successfully.
Book Added Successfully.
---------------------------------------------
---------------------
Book ID    Title                    Author
---------------------------------------------
```

Run Save

© 2026 **SIMATS Online Lab**. All rights reserved.

CO2-AT1-Scenario Based Assessment

Back

SIMATS Online Programming Lab

JAVA PROGRAMMING

MD Imthiaz I
192521360RD

20 marks

2/2 answered

registration number. The hospital management wants a Java application that maintains patient records efficiently while preventing duplicate registrations.

Each patient record consists of

Patient ID

Patient Name

Mobile Number

Department

Doctor Name

The system should utilize Java Collection Framework and String manipulation techniques for storing, validating, searching, and displaying patient information.

Functional Requirements

Develop a Java program using ArrayList and HashMap to perform the following operations.

Menu

1. Register Patient
2. Search Patient

3. Display All Patients
4. Count Department-wise Patients
5. Exit

Requirement 1 – Register Patient (4 Marks)
Accept
Patient ID
Patient Name
Mobile Number
Department
Doctor Name
Perform validations.
Patient ID should be unique.
Patient Name should contain only alphabets.
Mobile Number should contain exactly 10 digits.
Department should automatically convert to uppercase.
Remove unnecessary spaces from all string inputs.
Store the patient in
ArrayList
HashMap (PatientID as Key)

Requirement 2 – Search Patient (4 Marks)
Accept Patient ID.
Display complete patient details.
If not found,
Patient Not Found

Requirement 3 – Display All Patients (4 Marks)
Display in the following format.

ID Name Mobile Department Doctor

P101 Rahul 9876543210 CARDIOLOGY Dr.Arun
P102 Priya 9876543211 ORTHO Dr.Meena

Requirement 4 – Department-wise Count (4 Marks)
Count the number of patients in each department.
Example
CARDIOLOGY : 5

ORTHO : 3

NEURO : 4

Use

HashMap<String,Integer>

Requirement 5 – String Operations (4 Marks)

For every patient,

Perform

Convert Name to Proper Case

Remove leading/trailing spaces

Display Name Length

Display Department in Uppercase

Example

Input

raHUl kumAr

Output

Rahul Kumar

Length : 11

Department : CARDIOLOGY

Constraints

Maximum Patients = 100

Duplicate Patient IDs are not allowed.

Collections must be used.

Arrays are not allowed.

No Database.

Use only Java Collections.

Sample Input

1

P101

Rahul Kumar

9876543210

Cardiology

Dr.Arun

1

P102

Priya

9876543211

Ortho

Dr.Meena

3

4

2

P101

5

Sample Output

Patient Registered Successfully.

Patient Registered Successfully.

ID Name Department

P101 Rahul Kumar CARDIOLOGY
P102 Priya ORTHO

Department Count

CARDIOLOGY : 1

ORTHO : 1

Patient Details

ID : P101

Name : Rahul Kumar

Mobile : 9876543210

Department : CARDIOLOGY

Doctor : Dr.Arun

Your Code

```
import java.util.*;

```

```
public class Main {
    private static String toProperCase(St
        String[] words = name.trim().spli
        StringBuilder properName = new St
        for (int i = 0; i < words.length;
            if (!words[i].isEmpty()) {
                properName.append(Charact
                          .append(words[i
                if (i < words.length - 1)
                    properName.append(" "
                }
            }
        }
        return properName.toString();
    }

    public static void main(String[] args
        Scanner sc = new Scanner(System.i

        ArrayList<Patient> patientList =
        HashMap<String, Patient> patientM

        while (sc.hasNextLine()) {
            String choiceLine = sc.nextLi
            if (choiceLine.isEmpty()) cor

            int choice;
            try {
                choice = Integer.parseInt
            } catch (NumberFormatExceptic
                continue;
            }

            if (choice == 1) {
                if (!sc.hasNextLine()) br
                String id = sc.nextLine()

                if (!sc.hasNextLine()) br
                String name = sc.nextLine

                if (!sc.hasNextLine()) br
                String mobile = sc.nextLi

                if (!sc.hasNextLine()) br
                String department = sc.ne

                if (!sc.hasNextLine()) br
                String doctor = sc.nextLi
```

```java

                if (patientMap.containsKe
                    System.out.println("P
                    continue;
                }

                name = toProperCase(name)
                Patient p = new Patient(i
                patientList.add(p);
                patientMap.put(id, p);

                System.out.println("Patie

            } else if (choice == 2) {
                if (!sc.hasNextLine()) br
                String searchId = sc.next

                if (patientMap.containsKe
                    Patient p = patientMa
                    System.out.println("P
                    System.out.println("I
                    System.out.println("N
                    System.out.println("M
                    System.out.println("D
                    System.out.println("D
                } else {
                    System.out.println("P
                }

            } else if (choice == 3) {
                System.out.println("-----
                System.out.printf("%-7s %
                System.out.println("-----
                for (Patient p : patientL
                    System.out.printf("%-
                }

            } else if (choice == 4) {
                System.out.println("Depar
                HashMap<String, Integer>
                for (Patient p : patientL
                    deptCount.put(p.depar
                }
                for (Map.Entry<String, In
                    System.out.println(en
                }

            } else if (choice == 5) {
```

```
                    break;
                }
            }
        sc.close();
    }
}
class Patient {
    String id;
    String name;
    String mobile;
    String department;
    String doctor;

    public Patient(String id, String name
        this.id = id;
        this.name = name;
        this.mobile = mobile;
        this.department = department;
        this.doctor = doctor;
    }
}
```

Input

```
1
P101
Rahul Kumar
9876543210
```

Output

```
Patient Registered Successfully.
Patient Registered Successfully.
-----------------------------------------
-----------
ID      Name            Department
-----------------------------------------
```

Run Save

© 2026 **SIMATS Online Lab**. All rights reserved.